Katolicki Uniwersytet Lubelski Jana Pawła II

Wydział Filozofii

Instytut Filozofii

Sztuczna inteligencja, studia stacjonarne I stopnia

Jan Zdyb

Nr albumu: 162334

Próba automatycznej klasyfikacji cięciw według sposobów ich losowania w paradoksie Bertranda

Praca licencjacka
napisana na seminarium **Bazy wiedzy i automatyczne wnioskowanie**
pod kierunkiem
prof. dr hab. Piotr Kulicki, prof. KUL

Lublin 2026

Spis treści

1 Wstęp **5**

2 Teoretyczne omówienie paradoksu Bertranda i jego rozwiązań **7**
2.1 Rozwiązania przedstawione przez Bertranda 7
2.2 Rozwiązanie Jaynesa i krytyka Drory'ego 8
2.2.1 Rozwiązanie Jaynesa 8
2.2.2 Krytyka Drory'ego 9
2.3 Inne próby rozwiązania paradoksu 9
2.3.1 Aerts i Sassoli de Bianchi 9
2.3.2 Pavel Brazhnikov 9

3 Pozyskanie danych i ich walidacja **11**
3.1 Metoda 1 11
3.2 Metoda 2 13
3.3 Metoda 3 15
3.4 Zaprezentowanie wylosowanych cięciw na wykresach 17

4 Klasyfikacja cięciw za pomocą modeli sztucznej inteligencji **21**
4.1 Podział cięciw ze względu na metodę losowania 21
4.1.1 Las losowy - klasyfikowanie tylko na metody 22
4.1.2 Sieć neuronowa - klasyfikowanie tylko na metody 23
4.1.3 Podsumowanie dzielenia ze względu na metodę losowania 25
4.2 Podział cięciw ze względu na metodę losowania i długość cięciw 25
4.2.1 Las losowy - klasyfikowanie na długość i metodę 26
4.2.2 Sieć neuronowa - klasyfikowanie na długość i metodę 27
4.2.3 Podsumowanie dzielenia ze względu na długość i metodę losowania 29
4.3 Podział cięciw z uwzględnieniem końcowych elementów danych 30
4.3.1 Las losowy - klasyfikacja z użyciem końcówek 31
4.3.2 Sieć neuronowa - klasyfikacja z użyciem końcówek 33
4.3.3 Podsumowanie dzielenia z użyciem końcówek 34

5 Podsumowanie **35**

Rozdział 1

Wstęp

Paradoks Bertranda to jedno z klasycznych zagadnień teorii prawdopodobieństwa, które od lat budzi zainteresowanie matematyków, filozofów oraz badaczy zajmujących się podstawami rachunku prawdopodobieństwa. Został on sformułowany w XIX wieku przez Josepha Bertranda i dotyczy pozornie prostego pytania o prawdopodobieństwo, tego że losowo wybrana cięciwa w okręgu będzie dłuższa niż bok wpisanego w ten okrąg trójkąta równobocznego. Problem ten ujawnia jednak głębokie trudności związane z pojęciem losowości i sposobem definiowania losowego wybory w przestrzeni geometrycznej.

W niniejszej pracy przyjrzę się szczegółowo paradoksowi Bertranda, wyjaśniając na czym polega jego problem oraz jakie niesie ze sobą konsekwencje teoretyczne. Przedstawię trzy klasyczne rozwiązania zaproponowane przez samego Bertranda, z których każde opiera się na innej interpretacji losowego "wyboru cięciwy" i prowadzi do różnych wyników. Omówię również inne podejścia i ujęcia tego paradoksu, które pojawiły się w literaturze naukowej.

Główną część pracy poświęcona będzie jednak zastosowaniu metod informatycznych do analizy, próby rozwiązania tego paradoksu jak i wyciągnięcia na tej podstawie wniosków. W szczególności zaprezentuję skrypt symulujący losowe wybory cięciw oraz zestawię otrzymane wyniki z teoretycznymi przewidywaniami w celu walidacji skryptu. Następnie zaprezentuje kilka prostych modeli SI, których celem będzie klasyfikowanie uzyskanych przez nas losowych cięciw na metody za pomocą których zostały one stworzone, oraz porównanie tego jak owe modele poradziły sobie z tym nietypowym i z ludzkiej perspektywy niemalże niewykonalnym zadaniem. Skrypty te zostały stworzone za pomocą języka Python 3.12 w środowisku Visual Studio Code w notatniku Jupyter-a.

Celem tej pracy jest nie tylko zbadanie paradoksu z różnych punktów widzenia, ale także pokazanie, że narzędzia informatyczne - zarówno klasyczne algorytmy, jak i proste modele sztucznej inteligencji - mogą stanowić wartościowe uzupełnienie klasycznego podejścia matematycznego. Przedstawię również zalety i ograniczenia tego typu podejść, zwłaszcza w kontekście problemów probabilistycznych, gdzie intuicja często zawodzi, a precyzyjne definicje mają kluczowe znaczenie.

W rozdziale "Teoretyczne omówienie paradoksu Bertranda i jego rozwiązań" zostanie wyjaśniona istota paradoksu, jak powstał oraz różne próby jego rozwiązania. W kolejnym rozdziale zaprezentuję sposób tworzenia zbioru danych (dataset) oraz pokażę, że każda z przedstawionych metod Bertranda może zostać odtworzona w środowisku komputerowym, co zwaliduję zbiór danych. Po czym w rozdziale "Klasyfikacja cięciw za pomocą modelów sztucznej inteligencji" zaprezentuję stworzone przez siebie modele sztucznej inteligencji jak i ich wyniki.

Całość ma na celu pokazać, że modele sztucznej inteligencji są w stanie nauczyć się subtelnych różnic w dużych zbiorach danych, które byłyby nie do wyłapania przez ludzi.

Rozdział 2

Teoretyczne omówienie paradoksu Bertranda i jego rozwiązań

Paradoks Bertranda to paradoks w klasycznej interpretacji teorii prawdopodobieństwa z powodu prowadzenia do wyników z sobą sprzecznych w oparciu o założenia. Joseph Bertrand przedstawił go w swojej pracy "Calcul des probabités" z 1889 roku jako przykład ilustrujący, że zasada obojętności (znana również jako zasada równego prawdopodobieństwa) może prowadzić do niejednoznacznych wyników, jeśli zastosowana zostanie bezkrytycznie w przypadku przestrzeni nieskończonych możliwości, co wedle Bertranda ilustruje ograniczenia podejść czysto statystycznych.

2.1 Rozwiązania przedstawione przez Bertranda

Bertrand przedstawił trzy możliwe rozwiązania przedstawionego przez siebie problemu prawdopodobieństwa wylosowania cięciwy dłuższej niż bok trójkąta równobocznego wpisanego w okrąg, są to następujące rozwiązania [1]:

Metoda 1 - losowe końce cięciwy

W tej metodzie losowane są po kolei dwa punkty na okręgu, które następnie łączy się cięciwą. Aby obliczyć prawdopodobieństwo, że taka cięciwa jest dłuższa od boku trójkąta równobocznego wpisanego w okrąg, można stworzyć ten trójkąt tak aby jeden z jego wierzchołków pokrywał się z pierwszym wylosowanym przez nas punktem na okręgu.

Cięciwa będzie dłuższa od bok trójkąta wtedy i tylko wtedy, gdy drugi koniec cięciwy znajdzie się na łuku wyznaczonym przez dwa pozostałe wierzchołki trójkąta. Łuk ten stanowi jedną trzecią obwodu okręgu. Ponieważ drugi punkt wybierany jest losowo z rozkładem jednostajnym na okręgu, prawdopodobieństwo, że losowa cięciwa będzie dłuższa od boku trójkąta równobocznego wpisanego w okrąg, wynosi $\frac{1}{3}$.

Metoda 2 - losowy punkt na promieniu

W tej metodzie losowo wybierany jest jeden promień okręgu, a następnie jeden punkt na tym promieniu. Tworzona jest następnie cięciwa prostopadła do promienia przechodząca przez punkt na promieniu.

W trójkącie równobocznym wpisanym w okrąg odległość środka okręgu od każdego boku wynosi $\frac{R}{2}$, gdzie R jest promieniem okręgu. Oznacza to, że bok trójkąta przecina każdy promień w jego połowie. Ponieważ punkty na promieniu wybierane są w sposób losowy z rozkładem jednostajnym, prawdopodobieństwo, że znajdzie się on w odległości mniejszej niż $\frac{R}{2}$ od środka okręgu, wynosi $\frac{1}{2}$.

Metoda 3 - losowy środek cięciwy

W tej metodzie losowany jest punkt wewnątrz dysku (okrąg wraz z swoim wnętrzem), który następnie traktowany jest jako środek cięciwy. Przez ten punkt prowadzona jest cięciwa o dowolnym kierunku, której środek pokrywa się z wylosowanym punktem.

Długość cięciwy zależy od odległości jej środka od środka dysku. Cięciwa jest dłuższa od boku trójkąta równobocznego wpisanego w okrąg wtedy i tylko wtedy, gdy jej środek znajduje się w odległości mniejszej niż $\frac{R}{2}$ od środka dysku, gdzie R oznacza promień dysku. Oznacza to, że cięciwa jest dłuższa od boku trójkąta, jak i że środek cięciwy musi należeć do mniejszego okręgu współśrodkowego o promieniu $\frac{R}{2}$.

Ponieważ punkt wybierany jest losowo z jednakowym prawdopodobieństwem w całym dysku, a pole powierzchni okręgu o promieniu $\frac{R}{2}$ stanowi jedną czwartą pola okręgu o promieniu R, prawdopodobieństwo, że losowa cięciwa będzie dłuższa od boku trójkąta równobocznego wpisanego w okrąg, wynosi $\frac{1}{4}$.

Podsumowanie trzech metod

Jak można zauważyć powyższe trzy metody różnią się między innymi sposobem traktowania cięciw będących średnicami. Można ten problem wyeliminować poprzez ujednolicenie zagadnień, tak aby wyeliminować średnicę z rozważań, bez wpływu na końcowe wyniki prawdopodobieństw. Zauważyć jednak trzeba, że w każdej z tych metod możliwe jest wylosowanie średnicy, w metodzie pierwszej losujemy z wszystkich możliwych cięciw (w tym z średnic), w metodzie drugiej wylosowanie średnicy możliwe jest gdy wylosowany punkt jest na środku okręgu, w trzeciej metodzie wylosowanie średnicy jest możliwe przez możliwość wylosowania środka okręgu, wtedy dowolna cięciwa jest średnicą.

Warto zauważyć, że każde z tych rozwiązań zaprezentowanych przez Bertranda korzysta głównie z statystyki i probabilistyki, a nie z ciągłego losowania cięciw i sprawdzania ich długości, jak i część z nich jest nieintuicyjna co stwarza problem z ich zaimplementowaniem do czysto praktycznego rozwiązania. Właśnie z tego powodu problem ten nazywamy paradoksem, gdzie to z pozornie prostych założeń dochodzimy do sprzecznych z sobą wyników, mimo logicznej poprawności każdego z nich. Sprawia to, że niemożliwym się staje zaimplementowanie ich w czysto niestatystycznych kodach, których zadaniem byłoby rozwiązanie tego paradoksu.

Co więcej każda cięciwa w każdej z metod ma teoretycznie taką samą szanse na bycie wylosowaną, ponieważ dla każdej cięciwy są losowane dokładnie dwie współrzędne na okręgu, a co za tym idzie dziedziny tych rozwiązań są z sobą tożsame. Utrudnia to ich jakościowy podział na klasy oparte na sposób ich wylosowania, co zostanie poruszone w rozdziale czwartym.

2.2 Rozwiązanie Jaynesa i krytyka Drory'ego

Jedną z najbardziej znanych prób rozwiązania paradoksu może się pochwalić Edwin Jaynes, przez wiele lat wielu uznawało jego rozwiązanie za poprawne, lecz w 2015 roku jego tok myślenia został podważony przez Alona Drory'ego.

2.2.1 Rozwiązanie Jaynesa

W swoim artykule "The Well-Posed Problem" z 1973 roku Edwin Jaynes przedstawił rozwiązanie paradoksu Bertranda, oparte na zasadzie "maksymalnej niewiedzy"(maximum ignorancie), zgodnie z nią nie należy wykorzystywać informacji, które nie zostały podane w treści problemu bezpośrednio. Zauważył, że w oryginalnym paradoksie Bertrand nie określa ani położenia, ani rozmiaru okręgu i argumentował, że każda obiektywna i jednoznaczna odpowiedz musi być "obojętna" względem rozmiaru i położenia. Czyli rozwiązanie musi być niezmiennicze względem skali i przesunięcia (ang.scale and translation invariant) [2].

Dla lepszego wyjaśnienia zilustrujmy to: załóżmy, że cięciwy są losowo rozmieszczane na okręgu o średnicy 5 - np. przez rzucanie prostych patyków i konwertowanie ich na cięciwy po przez przycinanie lub wydłużenie ich tak by pasowały do narysowanego przez nas wcześniej okręgu. Następnie wewnątrz naszego okręgu umieszczamy mniejszy okrąg, np. o średnicy 2,75. Wówczas rozkład cięciw w tym mniejszym okręgu powinien być taki sam, jak ograniczony rozkład cięciw z większego okręgu (po przez ponowne dopasowanie przez nas patyków). Jeśli teraz przesuniemy mniejszy okrąg w inne miejsce wewnątrz większego, rozkład powinien pozostać niezmieniony. Zauważyć można, że rozkład cięciw zmienia się w przypadku metody trzeciej: rozkład cięciw w mniejszym okręgu będzie jakościowo inny niż rozkład w dużym okręgu, co za tym idzie metoda trzecia nie spełnia wymagań przedstawionych przez Jaynesa.

To samo dotyczy metody pierwszej, choć trudniej jest to sobie wyobrazić. Okazuje się, że tylko metoda druga z losowaniem punktu na promieniu i rysowanie cięciwy prostopadłej do promienia jest zarówno niezminnicza względem skali, jak i przesunięcia, metoda trzecia jest natomiast niezminnicza tylko względem skali, a metoda pierwsza nie jest niezminnicza względem żadnego z przedstawionych przez Jaynesa przekształceń.

Jaynes nie ograniczył się jednak do analizowania niezmienniczości i oceniania metod rozwiązania paradoksu Bertranda. Zamiast tego wykorzystał równania całkowe opisujące te niezminniczości do bezpośredniego wyznaczenia rozkładu prawdopodobieństwa. W przypadku tego problemu równania te mają jednoznaczne rozwiązanie, i jest nim metoda druga ($\frac{1}{2}$), oznacza to, że Jaynes uznał tą metodę za poprawne rozwiązanie paradoksu.

2.2.2 Krytyka Drory'ego

Jednak w roku 2015 Alon Drory podważył tą konkluzję Jaynesa, napisał on w swoim artykule "Failure and Uses of Jaynes' Principle of Transformation Groups", że wszystkie metody Bertranda spełniają wymogi przedstawione przez Jaynesa. Zwrócił on uwagę, że matematyczna realizacja własności niezmienniczości nie jest jednoznaczna, lecz zależy od przyjętego modelu losowania. Drory pokazuje, że każde z trzech oryginalnych rozwiązań można wyprowadzić przy użyciu niezmienniczości względem obrotu, skali i przesunięcia. Wnioskuje więc, że zasada Jaynesa jest równie podatna na interpretację, jak sama zasada obojętności. Cytując samego Drory'ego "...wbrew twierdzeniu Jaynesa, każde z trzech klasycznych rozwiązań problemu Bertranda (i nawet więcej) można wyprowadzić za pomocą zasady grup transformacji, używając dokładnie tych samych symetrii - mianowicie niezmienniczości obrotu, skali i przesunięcia" [3].

Drory podkreślił, że sposób losowania punktów wpływa na to czy metoda będzie realizowała własności niezmienniczości, w przypadku metody pierwszej będzie to losowanie po przez wybór losowego punktu na obwodzie i kierunku wskazującego drugi punkt, w przypadku metody drugiej będzie to wcześniej wspomniane rzucenie patykiem, natomiast dla metody trzeciej będzie to rzut lotką do wnętrza koła.

2.3 Inne próby rozwiązania paradoksu

Przez ponad sto lat od przedstawienia paradoksu przez Bertranda wielu matematyków, filozofów i innych uczonych próbowało rozwiązać ten problem. Wierzę, że po przez pokazanie części z nich ukazane zostanie to jak bardzo skomplikowany problem kryje się za pytaniem "jakie jest prawdopodobieństwo wylosowania cięciwy dłuższej od boku trójkąta równobocznego wpisanego w okrąg".

2.3.1 Aerts i Sassoli de Bianchi

W swoim artykule z 2014 roku "Solving the hard problem of Bertrand's paradox" autorzy przyglądają się, jak sami to nazwali, "łatwemu problemowi" i "trudnemu problemowi".

"Łatwy problem" dotyczy sytuacji, w której procedura losowania cięciwy została jednoznacznie określona - np. przez wybór punktu na promieniu, środka cięciwy lub końców na okręgu. Dla każdego z tych przypadków można bez większych trudności wyznaczyć odpowiadające mu rozkłady prawdopodobieństwa i uzyskać konkretne wartości $\frac{1}{2}$, $\frac{1}{3}$, $\frac{1}{4}$.

"Trudny problem" polega natomiast na tym, że nie określono, w jaki sposób cięciwa ma być losowana. To właśnie niejednoznaczność procedury losowania sprawia według aiorów, że klasyczna zasada obojętności okazuje się niewystarczająca do rozwiązywania paradoksu.. Na tym problemie głównie skupili się Aersta i Sassoli de Bianchi.

Ich propozycja rozwiązania paradoksu opiera się na założeniu, że skoro nie ma podstaw do przyjęcia jednej konkretnej procedury losowania to należy uwzględnić wszystkie możliwe sposoby generowania losowych cięciw (nie tylko te przedstawione w tej pracy). Wprowadzili więc pojęcie "uniwersalnej średniej", czyli wartości oczekiwanej prawdopodobieństwa wyliczonej poprzez uśrednienie po wszystkich dopuszczalnych metodach losowania.

Oznacza to że, nie wybierają jednej konkretnej procedury losowania, lecz uwzględniają ich nieskończenie wiele (korzystają z , każdej metodzie przypisują równą wagę, ostateczna wartość prawdopodobieństwa, która nie wynika z arbitralnego wyboru jednej interpretacji zasady obojętności, lecz uogólnionego podejścia statystycznego. Warto zaznaczyć, że choć ich wynik wynosi $\frac{1}{2}$, więc pokrywa się z jednym z klasycznych rozwiązań Bertranda, to ich podejście opiera się na innej podstawie. Zamiast bowiem przyjmować konkretną procedurę losowania, autorzy uśredniają wyniki dla wszystkich możliwych metod, co może być zaletą, ich rozwiązanie nie zależy bowiem od arbitralnych założeń geometrycznych, lecz wynika z uogólnionej zasady maksymalizacji niewiedzy.

Oto ich formuła ($x_1 = \frac{1}{2},\ x_2 = \frac{3}{2}$):

$$\mathcal{P}(L > \sqrt{3}) = \left\langle \mathcal{P}\left(x \in \left[\tfrac{1}{2}, \tfrac{3}{2}\right] \mid \rho\right)\right\rangle^{\text{univ}} = \tfrac{1}{2}$$

Jak sami pisali "Wykazaliśmy, że po wprowadzeniu odpowiedniego formalizmu matematycznego dla uśrednienia po wszystkich możliwych "sposobach rzucania patyka na okrąg" prawdopodobieństwo, że losowa cięciwa będzie dłuższa niż bok wpisanego trójkąta równobocznego, wynosi $\frac{1}{2}$" [4].

2.3.2 Pavel Brazhnikov

W swoim artykule z roku 2023 Brazhnikov podejmuje się krytycznej analizie dwóch z trzech klasycznych rozwiązań paradoksu Bertranda, opierając się na ich właściwościach geometrycznych. Celem jego jest wskazanie, które

z klasycznych rozwiązań można uznać za bardziej spójne i obiektywne z punktu widzenia geometrii, a które nie spełniają tych wymogów [5].

Autor skoncentrował się na problemie niejednoznaczności definicji "losowej cięciwy" i na tym, że różne metody generowania cięciw dają różne wartości prawdopodobieństwa $\frac{1}{2}$, $\frac{1}{3}$, $\frac{1}{4}$. Analizuje on w szczególności dwa z tych podejść, które wedle niego są nieodpowiednio interpretowane lub stosowane.

W swoim artykule podkreśla, że klasyczne rozwiązania paradoksu Bertranda, dające wartości $\frac{1}{2}$ oraz $\frac{1}{3}$, choć szeroko omawiane, nie spełniają kryterium pełnej obiektywności oraz jednoznaczności z punktu widzenia geometrii.

Metoda pierwsza, dająca wynik $\frac{1}{3}$, polega na losowaniu dwóch punktów na okręgu i połączenia ich linią, tworząc cięciwę. Brazhnikov wskazuje, że podejście to nie uwzględnia odpowiednio symetrii przesunięć wewnątrz koła, co prowadzi do braku inwalidacji translacyjnej, innymi słowy, prawdopodobieństwo zależy od wybranej pozycji okręgu lub przesunięcia figury, co stoi w sprzeczności z intuicyjnym założeniem o "losowości" cięciwy.

Z kolei metoda druga dająca wynik $\frac{1}{2}$, chociaż zachowuje pewne cechy symetrii, to wedle Brazhnikova nadal nie zapewnia pełnej obiektywności - zwłaszcza w kontekście jednorodności wyboru punktów w całym obszarze koła. Powodem jest to, że losowanie odbywa się wzdłuż promienia, co asymetryzuje sposób generowania cięciw w porównaniu do pozostałych metod.

Trzecia metoda, która daje wartość $\frac{1}{4}$, jest natomiast najbardziej spójna i geometrycznie obiektywna wedle Brazhnikova. W tym podejściu losowany jest punkt wewnątrz koła (środek cięciwy), z rozkładem równomiernym na całej powierzchni koła. Dzięki temu wszystkie punkty wewnątrz figury traktowane są jednakowo, a cięciwy generowane na tej podstawie zachowują pełną symetrię i jednorodność, z tych właśnie powodów uznaje on właśnie odpowiedź $\frac{1}{4}$ za poprawną. Zauważyć też należy, że taka procedura losowania spełnia kryteria niezmienniczości względem translacji, rotacji oraz skalowania, co stanowi argument do uznania tej odpowiedzi za najbardziej obiektywne i konsekwentne rozwiązanie paradoksu.

Rozdział 3

Pozyskanie danych i ich walidacja

W tym rozdziale zostanie przedstawione w jaki sposób został stworzony zbiór danych wykorzystanych później do treningu modeli klasyfikujących. Zostanie także zaprezentowane to, że klasyczne rozwiązania paradoksu Bertranda mogą zostać przeniesione do przestrzeni informatycznej jak i zaprezentowane na wykresie. Wszystkie zaprezentowane w tym rozdziale skrypty można znaleźć w pliku generowanie_danych.ipynb, z zaznaczeniem, że odcinki zdegenerowane mimo, że możliwe w poniższych skryptach, szansa na ich wylosowanie jest znikoma i nie mająca wpływu na końcowy wynik, oraz że ziarno nie będzie ustawiane dla możliwości wielokrotnego uruchomienia skryptu bez obawy o powtarzalność wyników losowania. Inspiracją takiego podejścia jest praca Javier Rodrigo, Mariló López i Sagrario Lantarón, którzy to wykorzystali kod napisany w MATLAB i Octave do przeliczenia prawdopodobieństwa wylosowania różnych cięciw okręgu [6].

Wszystkie trzy metody do generowania cięciw wykorzystują następujące biblioteki Pythona:

- decimal - biblioteka służąca do precyzyjnych obliczeń dziesiętnych,
- random - standardowa biblioteka do generowania liczb losowych,
- math - zawiera podstawowe funkcje matematyczne, takie jak pierwiastki, logarytmy czy funkcje trygonometryczne,
- csv - moduł Pythona do odczytu i zapisu plików CSV.

Jak i następujące zmienne:

- getcontext().prec=50 - określa precyzję do 50 miejsc po przecinku,
- r=Decimal(1) - jest to promień okręgu dla którego będziemy losować wszystkie cięciwy,
- threshold=Decimal(3).sqrt() - jest to długość boku trójkąta równobocznego wpisanego w okrąg o r=1.

Przed jeszcze samym losowaniem należy zaznaczyć kilka problemów losowania czegokolwiek za pomocą komputerów:

- nie jest to prawdziwa losowość, a pseudolosowość,
- liczby prawdziwie losowe mogą mieć dowolną długość swych rozwinięć dziesiętnych, komputery jednakże nam na to nie pozwalają,

będąc świadomi tych ograniczeń dla każdej z metod losowane było trzykrotnie 1200000 cięciw.

3.1 Metoda 1

Do stworzenia cięciw dla metody 1 stworzona została następująca funkcja:

```
def metoda_1(n,count=0, filename="metoda_1_4.csv"):
    with open(filename, mode='w',newline='') as file:
        writer=csv.writer(file)
        writer.writerow(['x1','y1','x2','y2','length','is_longer'])
        for i in range(n):
```

```
            theta1=Decimal(random.uniform(0,2*math.pi))
            theta2=Decimal(random.uniform(0,2*math.pi))

            x1=r*Decimal(math.cos(float(theta1)))
            y1=r*Decimal(math.sin(float(theta1)))
            x2=r*Decimal(math.cos(float(theta2)))
            y2=r*Decimal(math.sin(float(theta2)))

            center_x=(x1+x2)/Decimal(2)
            center_y=(y1+y2)/Decimal(2)

            dx=x2-x1
            dy=y2-y1
            length=(dx**2+dy**2).sqrt()

            is_longer=int(length>threshold)
            if is_longer==1:
                count+=1
            if i==119 or i==1199 or i==11999 or i==119999:
                probability=Decimal(count)/Decimal(i+1)
                print(f"Prawdopodobieństwo dla {i+1} losowań to {probability}")
            writer.writerow([
                format(x1,'.50f'),
                format(y1,'.50f'),
                format(x2,'.50f'),
                format(y2,'.50f'),
                format(length,'.50f'),
                is_longer
            ])
        probability=Decimal(count)/Decimal(n)
        print(f"Prawdopodobieństwo: {probability}, cięciwy większe od boku trójkąta:
            {count}, wszystkie cięciwy: {n}")
    print(f"Zapisano {n} cięciw do pliku {filename}")
```

Gdzie wartościami wejściowymi są kolejno:

- n - zmienna mówiąca ile cięciw ma zostać wygenerowane,
- count - zmienna mówiąca ile wygenerowanych cięciw było dłuższych od boku trójkąta równobocznego wpisanego w okrąg,
- filename - zmienna mówiąca jak ma się nazywać plik csv, w którym będą przechowywane dane cięciw.

Skrypt będzie zapisywał współrzędne obu krańców cięciwy, współrzedne środka, jej długość oraz to czy była dłuższa od wcześniej ustalonego boku trójkąta.

Dla każdej tworzonej cięciwy losowane są dwa kąty theta (theta1, theta2), które następnie wyznaczają współrzędne jej krańców względem środka okręgu, pozwala nam to na obliczenie odległości między nimi za pomocą wzoru na odległość euklidesową (czyli za pomocą twierdzenia Pitagorasa):

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Z zaznaczeniem, że $x_2 - x_1$ obliczone zostaje przed tą operacją i przypisana zostaje zmiennej dx, analogicznie jest dla $y_2 - y_1$, jej wartość przypisana zostaje do zmiennej dy. Po obliczeniu długości cięciwy sprawdzane jest czy jest ona dłuższa od boku trójkąta wpisanego w okrąg, jeśli tak to zmienna count zwiększa się o jeden.

Gdy stworzona zostanie cięciwa 120, 1200, 12000, 120000 (cięciwa 120 zostanie stworzona dla i=119, ponieważ pierwsza cięciwa zostanie stworzona dla i=0) oraz ostatnia to wyliczane zostaje jaki procent dotychczasowych cięciw był dłuższy od boku trójkąta, a następnie informacja ta zostaje wyświetlona.

Oto wyniki trzech pełnych losowań cięciw dla metoda_1:

Tabela 1: Prawdopodobieństwa dla kolejnych liczby losowań oraz podsumowanie końcowe dla metoda_1

Plik	Liczba losowań	Prawdopodobieństwo / Podsumowanie
metoda_1_1.csv	120	0,325
	1 200	0,35
	12 000	0,33308333333333333333333333333333333333333333333333
	120 000	0,333525
	Końcowe podsumowanie Prawdopodobieństwo: 0,3334125 Cięciwy dłuższe od boku trójkąta: 400 095 Wszystkie cięciwy: 1 200 000	
metoda_1_2.csv	120	0,34166666666666666666666666666666666666666666666667
	1 200	0,345
	12 000	0,33591666666666666666666666666666666666666666666667
	120 000	0,33235833333333333333333333333333333333333333333333
	Końcowe podsumowanie Prawdopodobieństwo: 0,3335475 Cięciwy dłuższe od boku trójkąta: 400 257 Wszystkie cięciwy: 1 200 000	
metoda_1_3.csv	120	0,33333333333333333333333333333333333333333333333333
	1 200	0,33083333333333333333333333333333333333333333333333
	12 000	0,3355
	120 000	0,33359166666666666666666666666666666666666666666667
	Końcowe podsumowanie Prawdopodobieństwo: 0,3325575 Cięciwy dłuższe od boku trójkąta: 399 069 Wszystkie cięciwy: 1 200 000	

Na podstawie przedstawionych powyższych danych możemy określić, że wraz z zwiększającą się ilością wylosowanych cięciw ich procent będących dłuższych od boku trójkąta równobocznego wpisanego w okrąg zbliża się do przedstawionej przez Bertranda odpowiedzi, co zgadza się z oczekiwaniami teoretycznymi i przemawia za poprawnością tworzenia cięciw dla zbioru danych na podstawie pierwszej metody.

3.2 Metoda 2

Do stworzenia cięciw dla metody 2 stworzona została następująca funkcja:

```
def metoda_2(n,count=0, filename="metoda_2_4.csv"):
    with open(filename, mode='w',newline='') as file:
        writer=csv.writer(file)
        writer.writerow(['x1','y1','x2','y2','center_x','center_y','length','
            is_longer'])
        for i in range(n):
            rho=Decimal(str(random.uniform(0,float(r))))
            angle=Decimal(str(random.uniform(0,2*math.pi)))

            cx=rho*Decimal(math.cos(float(angle)))
            cy=rho*Decimal(math.sin(float(angle)))

            square=r**2-rho**2
            length=square.sqrt()
            r_len=(cx**2+cy**2).sqrt()
            if r_len==0:
                px,py=Decimal(1),Decimal(0)
            else:
                px=-cy/r_len
                py=cx/r_len

            x1=cx+px*length
            y1=cy+py*length
            x2=cx-px*length
```

```
            y2=cy-py*length

            length=length*2
            is_longer=int(length>threshold)
            if is_longer==1:
                count+=1
            if i==119 or i==1199 or i==11999 or i==119999:
                probability=Decimal(count)/Decimal(i+1)
                print(f"Prawdopodobieństwo dla {i+1} losowań to {probability}")
            writer.writerow([
                format(x1,'.50f'),
                format(y1,'.50f'),
                format(x2,'.50f'),
                format(y2,'.50f'),
                format(cx,'.50f'),
                format(cy,'.50f'),
                format(length,'.50f'),
                is_longer
            ])
        probability=Decimal(count)/Decimal(n)
        print(f"Prawdopodobieństwo: {probability}, cięciwy większe od boku trójkąta:
            {count}, wszystkie cięciwy: {n}")
    print(f"Zapisano {n} cięciw do pliku {filename}")
```

Zmienne wejściowe są takie same jak dla funkcji metoda_1, z powodu dużego podobieństwa do poprzedniej metody opisane zostaną tylko różnice.

Tak samo jak dla metoda_1 zapisywane są te same dane o cięciwach.

Zamiast losować dwa kąty, losowany będzie jeden (angle), odzwierciedlać on ma kierunek promienia okręgu na którym będzie losowany punkt będący środkiem wylosowanej cięciwy (rho). Na podstawie tych dwóch danych możliwym staje się obliczenie współrzędnych cx i cy za pomocą wzorów na zmianę współrzędnych biegunowych na kartezjańskie:

$$x = \rho \cos \varphi,$$
$$y = \rho \sin \varphi.$$

Mając te dane możliwym staje się obliczenie długości, jak i współrzędnych końców cięciwy. Niech środek cięciwy będzie oznaczony przez C=(cx,cy), którego odległość od środka okręgu wynosi

$$\rho = \sqrt{c_x^2 + c_y^2}.$$

Długość połowy cięciwy ℓ wyznacza się na podstawie twierdzenia Pitagorasa:

$$\ell = \sqrt{r^2 - \rho^2}.$$

Wektor promienia wprowadzony do punktu C ma postać:

$$\vec{r} = (c_x, c_y),$$

a cięciwa jest prostopadła do tego wektora, dlatego kierunek cięciwy przyjmujemy jako wektor prostopadły:

$$\vec{p} = (-c_y, c_x).$$

Po normalizacji otrzymujemy jednostkowy wektor kierunkowy cięciwy:

$$\hat{p} = \begin{cases} \left(-\frac{c_y}{\rho}, \frac{c_x}{\rho}\right), & \rho \neq 0, \\ (1, 0), & \rho = 0, \end{cases}$$

gdzie przypadek rho=0 odpowiada sytuacji, gdy środek cięciwy pokrywa się z środkiem okręgu - kierunek cięciwy jest wtedy dowolny. Współrzędne końców cięciwy wyznacza się poprzez przesunięcie punktu C wzdłuż wektora $\hat{p}$ o odległość ℓ:

$$P_1 = (c_x, c_y) + \ell \hat{p},$$
$$P_2 = (c_x, c_y) - \ell \hat{p},$$

czyli jawnie:

$$x_1 = c_x - \frac{c_y}{\rho}\,\ell, \quad y_1 = c_y + \frac{c_x}{\rho}\,\ell,$$
$$x_2 = c_x + \frac{c_y}{\rho}\,\ell, \quad y_2 = c_y - \frac{c_x}{\rho}\,\ell.$$

Otrzymane punkty P_1 i P_2 stanowią końce cięciwy przechodzącej przez punkt C i leżącej w całości wewnątrz okręgu.

Następnie obliczana jest całkowita długość cięciwy po przez pomnożenie połowy tej długości przez 2. Proces sprawdzanie długości cięciwy jak i obliczanie jaki procent cięciw jest dłuższy od boku trójkąta równobocznego wpisanego w okrąg jest taki sam jak w funkcji metoda_1.

Oto wyniki trzech pełnych losowań cięciw dla metoda_2:

Tabela 2: Prawdopodobieństwa dla kolejnych liczby losowań oraz podsumowanie końcowe dla metoda_2

Plik	Liczba losowań	Prawdopodobieństwo / Podsumowanie
metoda_2_1.csv	120	0,475
	1 200	0,4975
	12 000	0,4975833333333333333333333333333333333333333333333333
	120 000	0,4983666666666666666666666666666666666666666666666667
	Końcowe podsumowanie	Prawdopodobieństwo: 0,4996225 Cięciwy dłuższe od boku trójkąta: 599 547 Wszystkie cięciwy: 1 200 000
metoda_2_2.csv	120	0,55
	1 200	0,515
	12 000	0,5015833333333333333333333333333333333333333333333333
	120 000	0,4994166666666666666666666666666666666666666666666667
	Końcowe podsumowanie	Prawdopodobieństwo: 0,5004291666666666666666666666666666666666666666666667 Cięciwy dłuższe od boku trójkąta: 600 515 Wszystkie cięciwy: 1 200 000
metoda_2_3.csv	120	0,5083333333333333333333333333333333333333333333333333
	1 200	0,515
	12 000	0,49525
	120 000	0,49815
	Końcowe podsumowanie	Prawdopodobieństwo: 0,4998341666666666666666666666666666666666666666666667 Cięciwy dłuższe od boku trójkąta: 599 801 Wszystkie cięciwy: 1 200 000

Na podstawie przedstawionych powyższych danych możemy określić, że wraz z zwiększającą się ilością wylosowanych cięciw ich procent będących dłuższych od boku trójkąta równobocznego wpisanego w okrąg zbliża się do przedstawionej przez Bertranda odpowiedzi, co zgadza się z oczekiwaniami teoretycznymi i przemawia za poprawnością tworzenia cięciw dla zbioru danych na podstawie drugiej metody.

3.3 Metoda 3

Do stworzenia cięciw dla metody 3 stworzona została następująca funkcja:

```
def metoda_3(n,count=0, filename="metoda_3_4.csv"):
    with open(filename, mode='w',newline='') as file:
        writer=csv.writer(file)
        writer.writerow(['x1','y1','x2','y2','center_x','center_y','length','
            is_longer'])
        for i in range(n):
            d=Decimal(random.uniform(0,1)).sqrt()*r
            angle=Decimal(str(random.uniform(0,2*math.pi)))
```

```
            cx=d*Decimal(math.cos(float(angle)))
            cy=d*Decimal(math.sin(float(angle)))

            length=(r**2-d**2).sqrt()
            perp_angle=angle+Decimal(math.pi/2)

            dx=length*Decimal(math.cos(float(perp_angle)))
            dy=length*Decimal(math.sin(float(perp_angle)))

            x1=cx+dx
            y1=cy+dy
            x2=cx-dx
            y2=cy-dy

            length=length*2
            is_longer=int(length>threshold)
            if is_longer==1:
                count+=1
            if i==119 or i==1199 or i==11999 or i==119999:
                probability=Decimal(count)/Decimal(i+1)
                print(f"Prawdopodobieństwo dla {i+1} losowań to {probability}")
            writer.writerow([
                format(x1,'.50f'),
                format(y1,'.50f'),
                format(x2,'.50f'),
                format(y2,'.50f'),
                format(cx,'.50f'),
                format(cy,'.50f'),
                format(length,'.50f'),
                is_longer
            ])
        probability=Decimal(count)/Decimal(n)
        print(f"Prawdopodobieństwo: {probability}, cięciwy większe od boku trójkąta:
            {count}, wszystkie cięciwy: {n}")
    print(f"Zapisano {n} cięciw do pliku {filename}")
```

Zmienne wejściowe są takie same jak dla funkcji metoda_1 oraz metoda_2, z powodu dużego podobieństwa do poprzedniej metody opisane zostaną tylko różnice.

Funkcja metoda_3 również generuje cięciwy koła z zapisem współrzędnych środka i końców, lecz w przeciwieństwie do metoda_2, środek cięciwy losowany jest równomiernie po powierzchni koła, a nie liniowo wzdłuż promienia.

Sposób losowania jest następujący:

- $d = r\sqrt{U(0,1)}$ - odległość środka cięciwy od środka okręgu, co zapewnia równomierne rozmieszczenie po powierzchni koła,
- $\varphi \sim \mathcal{U}(0, 2\pi)$ - kąt kierujący promieniem.

Współrzędne środka cięciwy oblicza się jak wcześniej:

$$c_x = d\cos\varphi,$$
$$c_y = d\sin\varphi.$$

Połowa długości cięciwy ℓ również wynika z twierdzenia Pitagorasa:

$$\ell = \sqrt{r^2 - d^2}.$$

Różnica względem metoda_2 polega na sposobie wyznaczania kierunku cięciwy. Tutaj bezpośrednio stosuje się kąt prostopadły do promienia:

$$\varphi_\perp = \varphi + \frac{\pi}{2},$$

a końce cięciwy wyznaczane są jako:

$$x_1 = c_x + \ell \cos \varphi_\perp, \quad y_1 = c_y + \ell \sin \varphi_\perp,$$
$$x_2 = c_x - \ell \cos \varphi_\perp, \quad y_2 = c_y - \ell \sin \varphi_\perp.$$

Całkowita długość cięciwy jest również 2ℓ, a następnie sprawdzany jest warunek długości cięciwy względem boku trójkąta równobocznego wpisanego w okrąg.

Podsumowując: główna różnica między metoda_2 i metoda_3 leży w sposobie losowania środka cięciwy i wyznaczania końców:

- metoda_2 - losowanie liniowe od środka do brzegu, normalizacja wektora prostopadłego,
- metoda_3 - losowanie równomierne po powierzchni koła, użycie kąta prostopadłego.

Oto wyniki trzech pełnych losowań cięciw dla metoda_3:

Tabela 3: Prawdopodobieństwa dla kolejnych liczby losowań oraz podsumowanie końcowe dla metoda_3

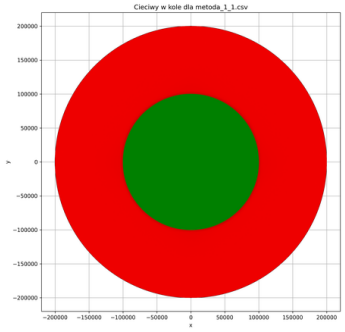
Plik	Liczba losowań	Prawdopodobieństwo / Podsumowanie
metoda_3_1.csv	120	0,308333333333333333333333333333333333333333333333333333
	1 200	0,263333333333333333333333333333333333333333333333333333
	12 000	0,253666666666666666666666666666666666666666666666666667
	120 000	0,251508333333333333333333333333333333333333333333333333
	Końcowe podsumowanie	
	Prawdopodobieństwo: 0,250229166666666666666666666666666666666666666666666667	
	Cięciwy większe od boku trójkąta: 400 149	
	Wszystkie cięciwy: 1 200 000	
metoda_3_2.csv	120	0,208333333333333333333333333333333333333333333333333333
	1 200	0,2475
	12 000	0,24325
	120 000	0,249408333333333333333333333333333333333333333333333333
	Końcowe podsumowanie	
	Prawdopodobieństwo: 0,249478333333333333333333333333333333333333333333333333	
	Cięciwy większe od boku trójkąta: 299 374	
	Wszystkie cięciwy: 1 200 000	
metoda_3_3.csv	120	0,25
	1 200	0,253333333333333333333333333333333333333333333333333333
	12 000	0,25175
	120 000	0,251458333333333333333333333333333333333333333333333333
	Końcowe podsumowanie	
	Prawdopodobieństwo: 0,250661666666666666666666666666666666666666666666666667	
	Cięciwy większe od boku trójkąta: 300 794	
	Wszystkie cięciwy: 1 200 000	

Na podstawie przedstawionych powyższych danych możemy określić, że wraz z zwiększającą się ilością wylosowanych cięciw ich procent będących dłuższych od boku trójkąta równobocznego wpisanego w okrąg zbliża się do przedstawionej przez Bertranda odpowiedzi, co zgadza się z oczekiwaniami teoretycznymi i przemawia za poprawnością tworzenia cięciw dla zbioru danych na podstawie trzeciej metody.

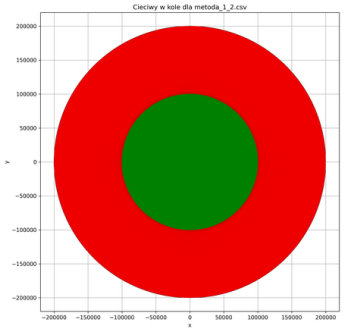
3.4 Zaprezentowanie wylosowanych cięciw na wykresach

Przedstawione poniżej wykresy zostały wygenerowane za pomocą bibliotek:

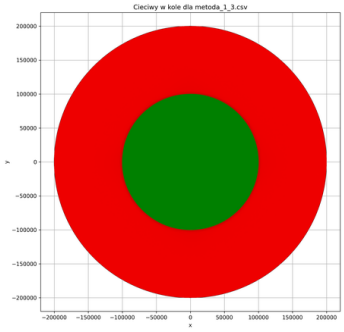
- csv - do wczytania danych z stworzonych wcześniej plików CSV,
- matplotlib.pyplot - do stworzenia wykresów.



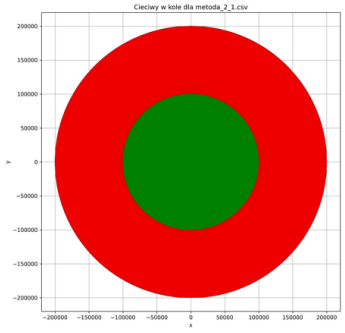
Rysunek 3.1: Wykres dla metoda_1_1



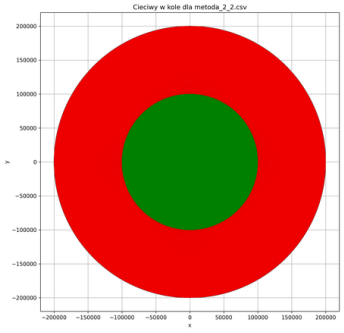
Rysunek 3.2: Wykres dla metoda_1_2



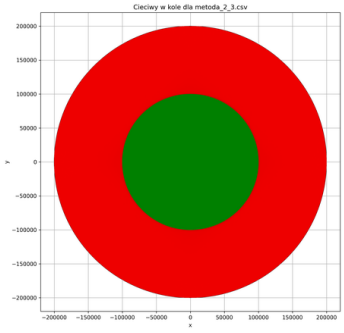
Rysunek 3.3: Wykres dla metoda_1_3



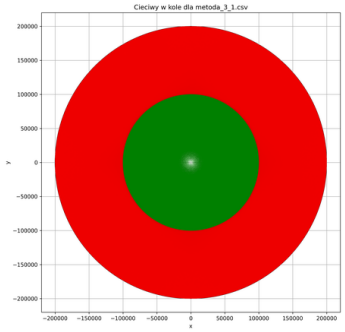
Rysunek 3.4: Wykres dla metoda_2_1



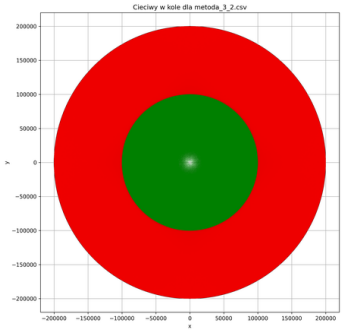
Rysunek 3.5: Wykres dla metoda_2_2



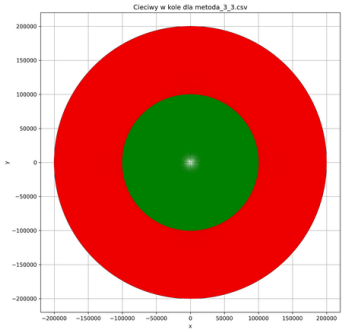
Rysunek 3.6: Wykres dla metoda_2_3



Rysunek 3.7: Wykres dla metoda_3_1



Rysunek 3.8: Wykres dla metoda_3_2



Rysunek 3.9: Wykres dla metoda_3_3

Skrypt na tworzenie wykresów znajduje się w pliku generowanie_danych.ipyb.

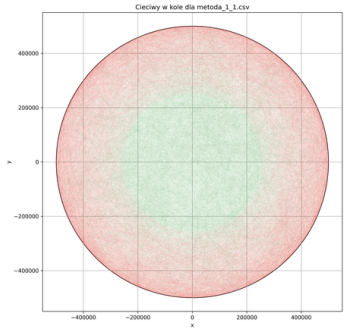
Poniższe wykresy przedstawiają rozkład cięciw w zależności od sposobu ich generowania, na zielonym zostały przedstawione cięciwy dłuższe od boku trójkąta wpisanego w okrąg, na czerwono równe lub krótsze od tej długości.

Metoda pierwsza i druga przy 1 200 000 cięciwach mają identyczne wykresy jak i wypełnione środki okręgu co za tym idzie duża część ich cięciw przechodzi w okolicach środka okręgu, czego nie można powiedzieć o wykresach reprezentujących metodę trzecią, gdzie to okolice środka okręgu nie są zapełnione co sugeruje o wiele mniejszą szansę na wylosowanie cięciwy blisko środka.

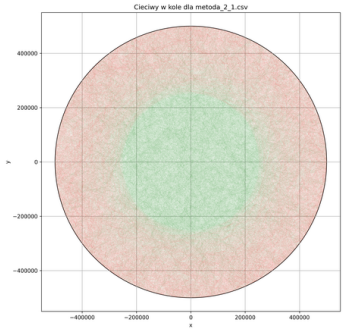
Nie widać większych różnic pomiędzy metodami generowania cięciw na powyższych wykresach, wynika to

z powodu ukazania każdej wylosowanej cięciwy w sposób pozwalający jej zaobserwowanie, gdyż cięciwy takie są dla nas normalnie niewidoczne z powodu ich nieskończenie małej grubości. Sprawia to, że przy większych ilościach cięciw (jak powyżej) cała powierzchnia zostaje wypełniona reprezentacjami cięciw.

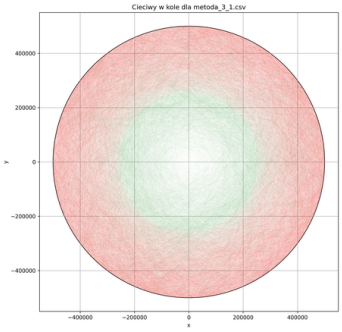
Dla zaprezentowania rozłożenia cięciw warto spojrzeć na wykresy pokazujące rozkład cięciw przy ich mniejszej ilości.



Rysunek 3.10: Wykres dla metoda_1_1 przy mniejszej ilości cięciw



Rysunek 3.11: Wykres dla metoda_2_1 przy mniejszej ilości cięciw



Rysunek 3.12: Wykres dla metoda_3_1 przy mniejszej ilości cięciw

Dzięki mniejszej ilości cięciw można zaobserwować następujące zjawiska:

- metoda pierwsza ma "zagęszczenie" cięciw przy samym brzegu okręgu,
- metoda druga ma najbardziej równomierne rozmieszczenie cięciw na powierzchni okręgu,
- metoda trzecia ma równomierne rozmieszczenie cięciw krótszych od boku trójkąta równobocznego wpisanego w okrąg z jednoczesną małą ilością cięciw blisko środka okręgu.

Rozdział 4

Klasyfikacja cięciw za pomocą modeli sztucznej inteligencji

W tym rozdziale zaprezentowane zostaną stworzone modele lasów losowych oraz sieci neuronowych na potrzeby automatycznej klasyfikacji cięciw według sposobu ich losowania, do treningu zostaną użyte wcześniej stworzone pliku CSV będące bazą danych.

Skrypty wykorzystane do stworzenia i wytrenowania modeli znajdują się w plikach model_las_losowy.ipynb oraz model_sieć_neuronowa.ipynb.

Do stworzenia modeli zostały wykorzystane następujące biblioteki:

- pandas - biblioteka do analizy i przetwarzania danych tabelarycznych. Udostępnia struktury DataFrame i Series, które pozwalają łatwo czyścić dane, filtrować je, agregować oraz przygotowywać do dalszej analizy lub uczenia modeli,
- numpy - biblioteka do obliczeń numerycznych w Pythonie. Umożliwia pracę na wielowymiarowych tablicach, szybkie operacje matematyczne i wektorowe, które są fundamentem większości obliczeń naukowych i uczenia maszynowego,
- glob - moduł do wyszukiwania plików na dysku za pomocą wzorców. Ułatwia automatyczne wczytywanie wielu plików jednocześnie,
- joblib - służy do zapisywania i wczytywania obiektów Pythona, szczególnie modeli uczenia maszynowego,
- matplotlib - arzędzie do wizualizacji danych w Pythonie, służy do tworzenia wykresów, histogramów oraz wizualizacji wyników modeli (np. macierz pomyłek),
- sklearn -biblioteka służąca do uczenia maszynowego. Zawiera gotowe algorytmy (np.Random Forest), narzędzia do podziału danych, skalowania cech oraz oceny jakości modeli,
- tensorflow - biblioteka do uczenia głębokiego. Umożliwia budowę i trenowanie sieci neuronowych oraz pracę na CPU i GPU,
- decimal.

Oraz wszystkie działania losowe będą tworzone na ziarnie 42 dla powtarzalności wyników jak i możliwości bardziej sprawiedliwego ich porównywania.

4.1 Podział cięciw ze względu na metodę losowania

Najprostszym i najszybszym sposobem jest klasyfikowanie tylko ze względu na metodę losowania. Przygotowanie danych do treningu modeli dla tego podejścia wygląda następująco:

```
method_1_files=glob.glob("../dane/metoda_1_*.csv")
method_2_files=glob.glob("../dane/metoda_2_*.csv")
method_3_files=glob.glob("../dane/metoda_3_*.csv")
```

```
def load_and_process(filepaths,method_label):
    dfs=[]
    for path in filepaths:
        df=pd.read_csv(path)
        df['distance_from_center']=np.sqrt(df['center_x']**2+df['center_y']**2)
        df['method']=method_label
        dfs.append(df)
    return pd.concat(dfs,ignore_index=True)

df1=load_and_process(method_1_files,method_label=0)
df2=load_and_process(method_2_files,method_label=1)
df3=load_and_process(method_3_files,method_label=2)
data=pd.concat([df1,df2,df3],ignore_index=True)

features=['x1','y1','x2','y2','length','center_x','center_y','distance_from_center']
X=data[features]
y=data['method']
```

Wykorzystana zostaje tutaj biblioteka glob do łatwego wczytania wszystkich plików CSV wchodzących w skład bazy danych, następnie dla każdej metody uruchamiana jest funkcja load_and_process, jej zadaniem jest przepisanie podanej części bazy danych w taki sposób aby dodać kolumnę będącą odległością środka okręgu od środka cięciwy oraz dodać kolumnę w której znajdzie się etykieta reprezentująca sposób wylosowania cięciwy z zaznaczeniem, że dla cieci neuronowej etykietami będą liczby 0,1,2, a dla lasu losowego 1,2,3.

Wynika to z sposobu ich działania, gdzie to sieć neuronowa zaczyna numerowanie klas od zera, a las losowy dzieli klasy w zależności od podanych przez nas nazw, oznacza to, że klasy podane przez sieć neuronową trzeba odkodować, a las losowy podaje nam odpowiedzi zgodnie z nazwami klas jakie mu podaliśmy.

Po tym procesie wszystkie dane są z sobą łączone, tworzona jest tablica przechowująca wszystkie wybrane do procesu treningu dane jak i tablica zawierająca etykiety.

4.1.1 Las losowy - klasyfikowanie tylko na metody

Las losowy jest jednym z najbardziej podstawowych i skutecznych sposobów klasyfikowania w uczeniu maszynowym. Dla przedstawionego problemu został stworzony w następujący sposób, wraz z wyświetleniem wyników klasyfikacji w macierzy pomyłek oraz zapisem modelu:

```
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)

model=RandomForestClassifier(n_estimators=100,random_state=42)
model.fit(X_train,y_train)

y_pred=model.predict(X_test)
acc=accuracy_score(y_test,y_pred)
print(f'Accuracy: {acc:.4f}')

cm=confusion_matrix(y_test,y_pred)
disp=ConfusionMatrixDisplay(confusion_matrix=cm,display_labels=['Metoda 1','Metoda 2'
    ,'Metoda 3'])
disp.plot()
plt.title('Macierz pomyłek dla modelu random_forest')
plt.savefig('macierz_pomyłek_dla_random_forest.png',dpi=300,bbox_inches='tight')
plt.show()

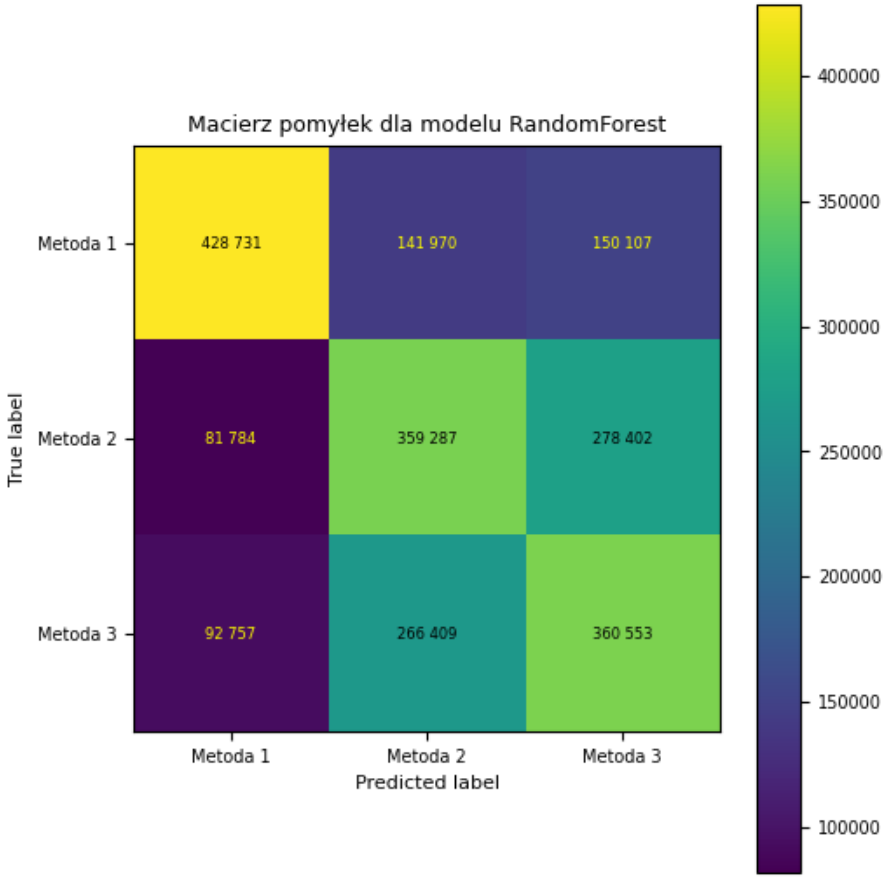
importance_of_features=model.feature_importances_
for f, imp in zip(features, importance_of_features):
    print(f'{f}: {imp:.3f}')
joblib.dump(model,'model_random_forest.pkl')
```

Las losowy składa się z stu różnych drzew decyzyjnych, dzięki temu błąd w pojedynczym drzewie nie będzie miał większego wpływu na pokazywane wyniki. Po treningu przeprowadzana jest walidacja modelu na stworzonym zbiorze walidacyjnym, a wyniki tej, że walidacji są wyświetlane w postaci macierzy pomyłek, następnie sprawdzana jest ważność cech, a sam model zapisany pod postacią model_random_forest.pkl.

Model ten osiągnął skuteczność na poziomie 53,16%, ważność cech wyglądała następująco:

- x1: 0.121,
- y1: 0.128,
- x2: 0.123,
- y2: 0.127,
- length: 0.121,
- center_x: 0.129,
- center_y: 0.128,
- distance_from_center: 0.123,

a macierz pomyłek wyglądała następująco:



Rysunek 4.1: Macierz pomyłek dla modelu random forest

Na podstawie powyższych wyników widać, że model ma rację w ponad połowie przypadków, wszystkie podane cechy mają podobny poziom ważności dla modelu, jak i model względnie dobrze radzi sobie z rozpoznawaniem metody pierwszej, myli natomiast z sobą metodę drugą i trzecią. Wyniki te mogą wynikać z podobnego procesu losowania cięciw w metodzie drugiej i trzeciej.

4.1.2 Sieć neuronowa - klasyfikowanie tylko na metody

Sieci neuronowe są najbardziej wszechstronnym rodzajem modeli w dziedzinie sztucznej inteligencji i uczenia maszynowego. Zawdzięczają to w dużej części biologicznym neuronom na których są one wzorowane. Podobnie jak lasy losowe one także są szeroko stosowane w zadaniach wymagających klasyfikowania na podstawie podanych

cech. Oto stworzony model którego zadaniem jest dzielenie cięciw na podstawie ich cech na metody za pomocą której została ona stworzona:

```
scaler=StandardScaler()
X_scaled=scaler.fit_transform(X)
y_cat=tf.keras.utils.to_categorical(y,num_classes=3)

X_train,X_test,y_train,y_test=train_test_split(X_scaled,y_cat,test_size=0.2,
    random_state=42)

model=Sequential([
    Dense(64,activation='relu',input_shape=(X_train.shape[1],)),
    Dropout(0.2),
    Dense(32,activation='relu'),
    Dropout(0.2),
    Dense(3,activation='softmax')
])

model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

history=model.fit(X_train,y_train, epochs=20,batch_size=32,validation_split=0.2,
    verbose=1,random_state=42)

loss,accuracy=model.evaluate(X_test,y_test,verbose=0)
print(f'Dokładność na danych testowych: {accuracy:.4f}')

y_pred_probs=model.predict(X_test)
y_pred=np.argmax(y_pred_probs,axis=1)
y_true=np.argmax(y_test,axis=1)

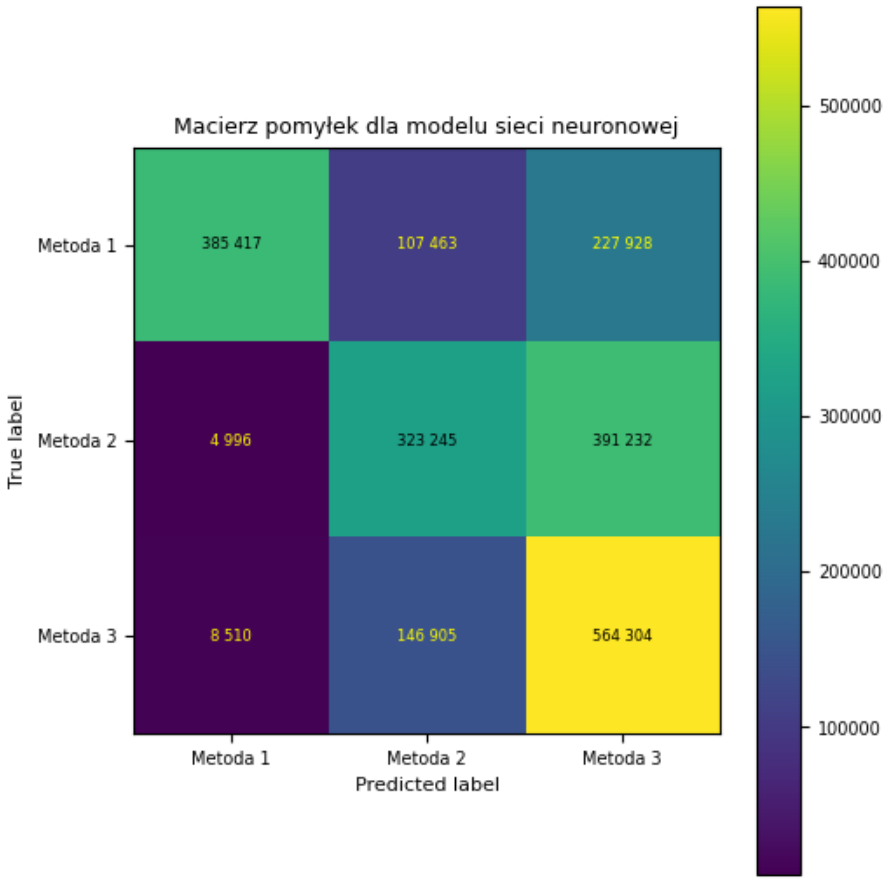
cm=confusion_matrix(y_true,y_pred)
disp=ConfusionMatrixDisplay(confusion_matrix=cm,display_labels=['Metoda 1','Metoda 2'
    ,'Metoda 3'])
fig,ax=plt.subplots(figsize=(6,6))
disp.plot(ax=ax)
plt.title('Macierz pomyłek dla modelu sieć_neuronowa')
plt.savefig('macierz_pomyłek_dla_sieć_neuronowa.png',dpi=300)
plt.show()
model.save("model_cieć_neuronowa.h5")
joblib.dump(scaler, "scaler_do_model_cieć_neuronowa.pkl")
print("Model i scaler zapisane.")
```

Na początku dane są skalowane dla lepszego działania tworzonej sieci, ponieważ sieci najlepiej sobie radzą gdy wszystkie dane są ustandaryzowane. Następnie po podziale danych na zbiór treningowy i walidacyjny definiowana jest sekwencyjna sieć neuronowa z użyciem warstw Dense (warstwy gęste) jak i warstw Dropout które mają za zadanie "wyłączyć"procent losowych neuronów dla zmniejszej niezależności od pojedynczych neuronów, a podczas jej kompilacji ustawiany jest optymizer na "adam".

Model zostanie zapisany w dwóch plikach:

- model_cieć_neuronowa.h5 - wytrenowana sieć neuronowa,
- scaler_do_model_cieć_neuronowa.pkl - obiekt skalera danych.

Sieć ta po treningu osiągnęła skuteczność na poziomie 58,86%, a jej macierz pomyłek wygląda następująco:



Rysunek 4.2: Macierz pomyłek dla modelu sieci neuronowej

Widać, że model osiąga skuteczność 58,86%, najlepiej sobie radzi z rozpoznawaniem metody trzeciej i nie myli jej z metodą drugą, jednakże metodę drugą z trzecią już tak. Metoda pierwsza natomiast jest rozpoznawana w ponad połowie przypadków.

4.1.3 Podsumowanie dzielenia ze względu na metodę losowania

Jak można było zaobserwować sieć neuronowa radzi sobie lepiej (o około 5%), jednocześnie lepiej daje ona sobie radę z klasyfikowaniem metody trzeciej, las losowy natomiast lepiej sobie poradził z rozpoznawaniem metody pierwszej. Oba modele jednakże, nie poradziły sobie z rozpoznawaniem metody drugiej która często była klasyfikowana jako metoda trzecia.

4.2 Podział cięciw ze względu na metodę losowania i długość cięciw

Podejście to jest bardzo podobne do poprzedniego, lecz zamiast mieć trzy klasy, po jednej dla każdej metody, każda klasa będzie miała dwie, pierwszą dla cięciw będących krótszymi lub równymi bokowi trójkąta równobocznego (krótkie), drugą dla cięciw będących dłuższymi od boku wspomnianego trójkąta.

Oba modele korzystają z następującego wczytania danych:

```
method_1_files=glob.glob("../dane/metoda_1_*.csv")
method_2_files=glob.glob("../dane/metoda_2_*.csv")
method_3_files=glob.glob("../dane/metoda_3_*.csv")

def load_and_process(filepaths,method_label):
    dfs=[]
    for path in filepaths:
        df=pd.read_csv(path)
```

```
        df['distance_from_center']=np.sqrt(df['center_x']**2+df['center_y']**2)
        df['method']=method_label
        df['class']=(method_label-1)*2+df['is_longer']+1
        dfs.append(df)
    return pd.concat(dfs,ignore_index=True)

df1=load_and_process(method_1_files,method_label=1)
df2=load_and_process(method_2_files,method_label=2)
df3=load_and_process(method_3_files,method_label=3)
data=pd.concat([df1,df2,df3],ignore_index=True)

features=['x1','y1','x2','y2','length','center_x','center_y','distance_from_center']
X=data[features]
y=data['class']
```

Jedyną zmianą w porównaniu do wcześniejszego wczytania klas jest

```
df['class']=(method_label-1)*2+df['is_longer']+1
```

w funkcji load_and_process dzięki której tworzone jest sześć klas, a nie trzy jak wcześniej.

4.2.1 Las losowy - klasyfikowanie na długość i metodę

Model lasu losowego podobnie jak w poprzednim wypadku też korzystać będzie z stu drzew decyzyjnych i nie przeszedł, żadnych większych zmian oprócz zwiększenia ilości etykiet oraz zmiany nazwy modelu na model_random_forest_metoda_i_długość_cięciwy.pkl. Oto jak wygląda skrypt:

```
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)

model=RandomForestClassifier(n_estimators=100,random_state=42)
model.fit(X_train,y_train)

y_pred=model.predict(X_test)
acc=accuracy_score(y_test,y_pred)
print(f'Accuracy: {acc:.4f}')

labels=[1,2,3,4,5,6]
label_names=['Metoda 1-krótka','Metoda 1-długa','Metoda 2-krótka','Metoda 2-długa','
    Metoda 3-krótka','Metoda 3-długa']

cm=confusion_matrix(y_test,y_pred,labels=labels)
disp=ConfusionMatrixDisplay(confusion_matrix=cm,display_labels=label_names)
disp.plot()
plt.title('Macierz pomyłek dla modelu random_forest, metoda i długość cięciwy')
plt.tight_layout()
plt.savefig('macierz_pomyłek_dla_random_forest_metoda_i_długość_cięciwy.png',dpi=300)
plt.show()

importance_of_features=model.feature_importances_
for f, imp in zip(features, importance_of_features):
    print(f'{f}: {imp:.3f}')
joblib.dump(model,'model_random_forest_metoda_i_długość_cięciwy.pkl')
```

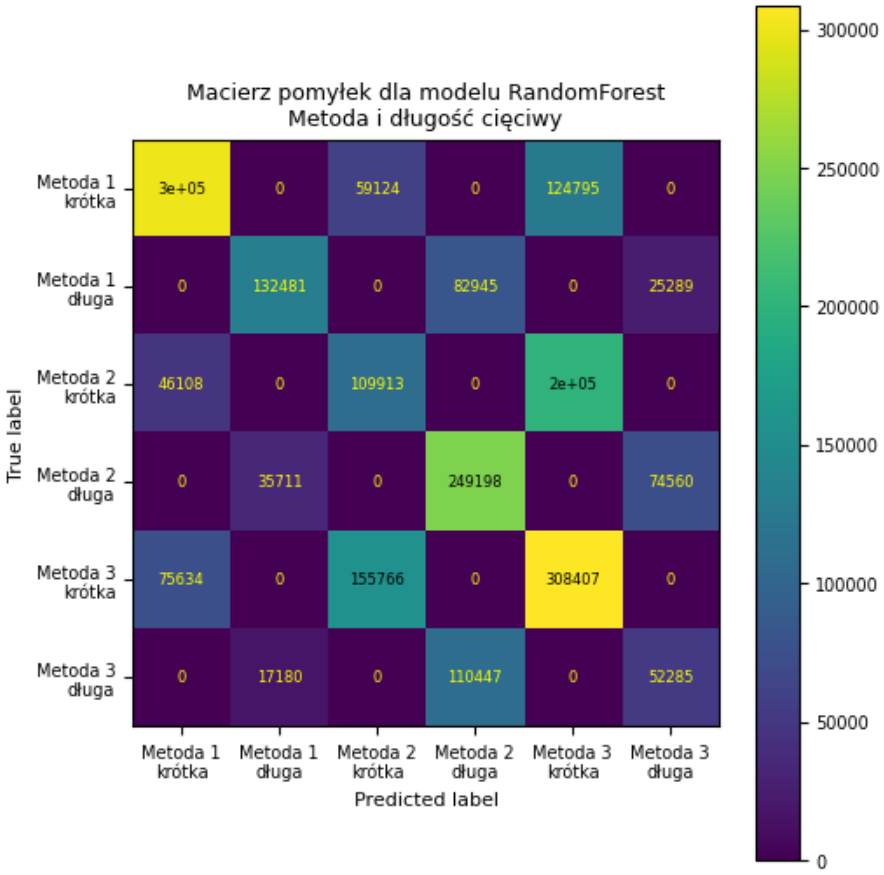
Ten las losowy osiągną skuteczność na poziomie 53,15%, co stanowi niemalże taką samą skuteczność co wcześniejszy las losowy dzielący cięciwy tylko na klasy bez dodatkowego podziału na długości co oznacza, że dla drzew decyzyjnych wchodzących w skład lasu losowego decyzja o tym czy cięciwa jest dłuższa od boku trójkąta równobocznego wpisanego w okrąg jest dość łatwa.

Ważność cech w przypadku tego zadania wyglądała następująco:

- x1: 0.103,
- y1: 0.103,
- x2: 0.101,

- y2: 0.105,
- length: 0.174,
- center_x: 0.113,
- center_y: 0.113,
- distance_from_center: 0.187,

a macierz pomyłek wyglądała następująco:



Rysunek 4.3: Macierz pomyłek dla modelu random forest z uwzględnieniem podziału na cięciwy dłuższe i krótsze

Na podstawie powyższych wyników widać, że model ma rację w ponad połowie przypadków, cechy length oraz distance_from_center są wyraźnie ważniejsze dla modelu w porównaniu z pozostałymi cechami oraz w porównaniu do ważności cech dla model_random_forest.pkl.

Model ten bardzo dobrze rozpoznaje cięciwy krótkie metody pierwszej oraz cięciwy krótkie metody trzeciej, jak i cięciwy długie metody drugiej. Myli on natomiast cięciwy długie metody trzeciej z cięciwami długimi metody drugiej jak i cięciwy krótkie metody drugiej z cięciwami krótkimi metody trzeciej. Może to wynikać z powodu prawdopodobieństwa wylosowania cięci o konkretnych długościach w zależności od metody losowania i przypisywanie przynależności do klas w oparciu na nich.

4.2.2 Sieć neuronowa - klasyfikowanie na długość i metodę

Podobnie jak w przypadku model_random_forest_metoda_i_długość_cięciwy.pkl kod nie różni się zbytnio w porównaniu z poprzednią siecią neuronową model_sieć_neuronowa.h5. Jedyną zmianą jest dostosowanie sieci do większej ilości klas po przez jej powiększenie o jedną warstwę Dense oraz o jedną warstwę Dropout. Kod wygląda następująco:

```
scaler=StandardScaler()
X_scaled=scaler.fit_transform(X)
y_cat=tf.keras.utils.to_categorical(y-1,num_classes=6)

X_train,X_test,y_train,y_test=train_test_split(X_scaled,y_cat,test_size=0.2,
    random_state=42)

model=Sequential([
    Dense(64,activation='relu',input_shape=(X_train.shape[1],)),
    Dropout(0.2),
    Dense(32,activation='relu'),
    Dropout(0.2),
    Dense(16,activation='relu'),
    Dropout(0.2),
    Dense(6,activation='softmax')
])

model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

history=model.fit(X_train,y_train, epochs=20,batch_size=32,validation_split=0.2,
    verbose=1,random_state=42)

loss,accuracy=model.evaluate(X_test,y_test,verbose=0)
print(f'Dokładność na danych testowych: {accuracy:.4f}')

y_pred_probs=model.predict(X_test)
y_pred=np.argmax(y_pred_probs,axis=1)
y_true=np.argmax(y_test,axis=1)

labels=[0,1,2,3,4,5]
label_names=['Metoda 1-krótka','Metoda 1-długa','Metoda 2-krótka','Metoda 2-długa','
    Metoda 3-krótka','Metoda 3-długa']

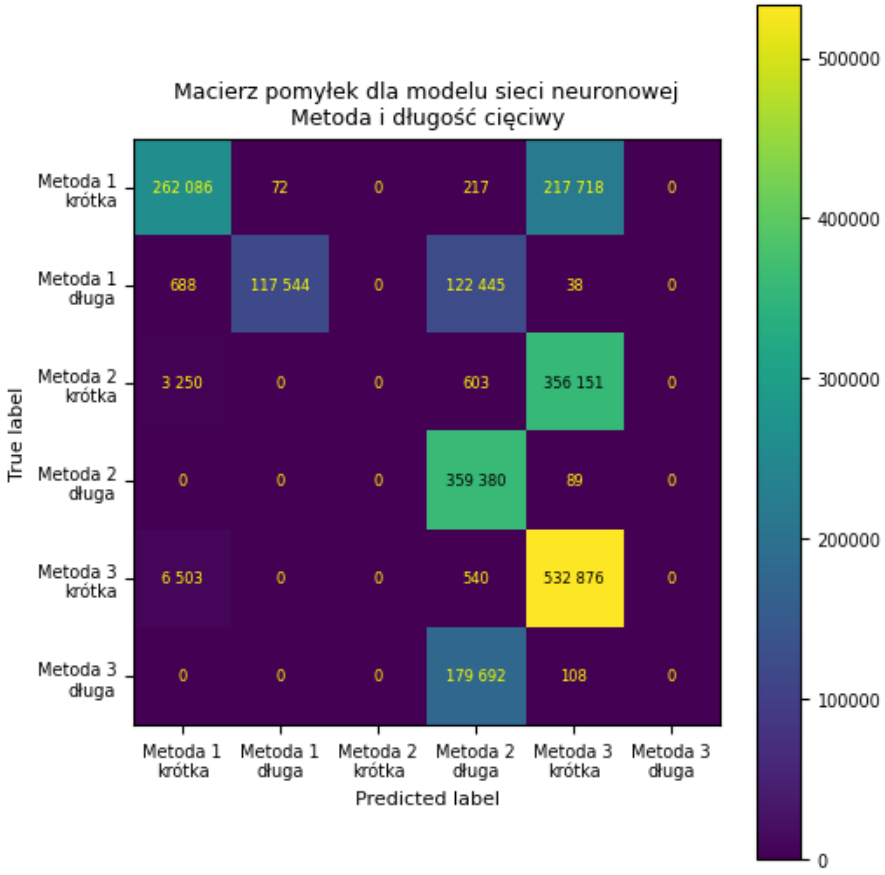
cm=confusion_matrix(y_true,y_pred,labels=labels)
disp=ConfusionMatrixDisplay(confusion_matrix=cm,display_labels=label_names)
disp.plot()
plt.title('Macierz pomyłek dla modelu sieci neuronowej, metoda i długość cięciwy')
plt.tight_layout()
plt.savefig('macierz_pomyłek_dla_sieci_neuronowej_metoda_i_długość_cięciwy.png',dpi
    =300)
plt.show()
model.save("model_cieć_neuronowa_metoda_i_długość_cięciwy.h5")
joblib.dump(scaler, "scaler_do_model_cieć_neuronowa_metoda_i_długość_cięciwy.pkl")
print("Model i scaler zapisane.")
```

Model zostaje zapisany w dwóch plikach:

- model_cieć_neuronowa_i_długość_cięciwy.h5 - wytrenowana sieć neuronowa,
- scaler_do_model_cieć_neuronowa_i_długość_cięciwy.pkl - obiekt skalera danych.

Sieć ta po treningu osiągnęła skuteczność na poziomie 58,60% co jest marginalnym pogorszeniem wyniku w porównaniu z model_cieć_neuronowa.h5 (58,86%).

Macierz pomyłek dla tej sieci neuronowej wygląda następująco:



Rysunek 4.4: Macierz pomyłek dla modelu sieci neuronowej z uwzględnieniem podziału na cięciwy dłuższe i krótsze

Widać, że model bardzo dobrze nauczył się rozpoznawać cięciwy krótkie metody trzeciej, jak i cięciwy długie metody drugiej lecz prawie wszystkie cięciwy długie metody trzeciej przypisuje metodzie długiej, a cięciwy krótkie metody drugiej przypisuje metodzie trzeciej. Wynika to najpewniej z szansy na powstanie cięciwy o konkretnych długościach w zależności od metody.

Dla metody pierwszej blisko połowa jej cięciw krótkich została przypisana metodzie trzeciej, podobna sytuacja występuje w przypadku jej cięciw długich gdzie lekko ponad połowa została przypisana metodzie drugiej. Wynika to najpewniej z szansy na powstanie cięciwy o konkretnych długościach w zależności od metody, lecz w porównaniu z metodą drugą i trzecią gdzie model przypisywał do każdego tylko jeden rodzaj cięciw to dla metody pierwszej podział procentowy poprawnie przypisanych cięciw jest zbliżony do prawdziwego wynoszącego $\frac{1}{3}$ cięciw dłuższych w stosunku do wszystkich cięciw.

4.2.3 Podsumowanie dzielenia ze względu na długość i metodę losowania

Jak można było zaobserwować sieć neuronowa znowu poradziła sobie z tym zadaniem lepiej od lasu losowego (o około 5%), gorzej rozpoznaje cięciwy metody pierwszej oraz przypisuje znaczną większość cięciw dłuższych od boku trójkąta równobocznego wpisanego w okrąg metodzie drugiej, a cięciwy krótsze metodzie trzeciej. Sieci zdarza się także błędnie określać czy cięciwa należy do klas zawierających cięciwy dłuższe lub krótsze, las losowy nie ma tego problemu.

Las losowy także podaje bardziej zróżnicowane wyniki, nie jest tak, że prawie wszystkie cięciwy krótsze są przypisywane do jednej metody, i wice versa. Las najlepiej rozpoznaje cięciwy krótkie metody pierwszej jak i cięciwy krótsze metody trzeciej.

4.3 Podział cięciw z uwzględnieniem końcowych elementów danych

Podczas przygotowania danych do treningu modeli sztucznej inteligencji wartości numeryczne były losowane z wysoką precyzją, a następnie rzutowane do formatu float32, wykorzystywanego przez biblioteki uczenia maszynowego. W praktyce oznacza to, że pierwotne dane zapisane z bardzo dużą precyzją miejsc po przecinku po skalowaniu i konwersji tracą znaczną część precyzji - do około 7 cyfr znaczących [8].

Aby ograniczyć negatywny wpływ tej utraty dokładności, zastosowane zostaje dodatkowe przetwarza danych polegające na rozdzieleniu każdej wartości na część główną (float64) oraz tzw. końcówkę, czyli różnicę pomiędzy wartością oryginalną, a jej reprezentacją zmiennoprzecinkową. Końcówki te zostają dodane do zbioru danych jako osobne cechy wejściowe modelu, co pozwala na zachowanie informacji o drobnych różnicach numerycznych pomijanych w standardowej reprezentacji float32.

Aby to osiągnąć dla obu modeli wykorzystany zostaje następujący skrytp:

```
method_1_files=glob.glob("../metoda_1_*.csv")
method_2_files=glob.glob("../metoda_2_*.csv")
method_3_files=glob.glob("../metoda_3_*.csv")

def load_and_process(filepaths,method_label):
    dfs=[]
    for path in filepaths:
        df=pd.read_csv(path,dtype=str)

        numeric_cols=['x1','y1','x2','y2','length']
        df=split_float64_and_tail(df,numeric_cols)
        df['distance_from_center']=np.sqrt(df['center_x']**2+df['center_y']**2)
        df['is_longer']=df['is_longer'].astype(int)
        df['method']=method_label
        df['class']=(method_label-1)*2+df['is_longer']+1
        dfs.append(df)
    return pd.concat(dfs,ignore_index=True)

def split_float64_and_tail(df,columns):
    for col in columns:
        floats=[]
        tails=[]
        for v in df[col]:
            dec=Decimal(v)
            f64=Decimal(str(np.float64(v)))
            tail=dec-f64
            floats.append(float(f64))
            tails.append(float(tail))

        df[col]=floats
        df[f'{col}_koncowka']=tails
    return df

getcontext().prec=50

df1=load_and_process(method_1_files,method_label=1)
df2=load_and_process(method_2_files,method_label=2)
df3=load_and_process(method_3_files,method_label=3)
data=pd.concat([df1,df2,df3],ignore_index=True)

base_features=['x1','y1','x2','y2','length','center_x','center_y','
    distance_from_center']
tail_features=[f'{f}_koncowka' for f in ['x1','y1','x2','y2','length','center_x','
    center_y']]
features=base_features+tail_features
X=data[features].astype(float)
y=data['class']
```

którego główną różnicą w porównaniu z wersją przygotowującą dane pod trening wcześniejszych modeli jest funkcja split_float64_and_tail której zadaniem jest wykonanie wcześniej opisanego procesu.

4.3.1 Las losowy - klasyfikacja z użyciem końcówek

Model lasu losowego będzie korzystał tylko z 50 drzew losowych, a nie stu jak w poprzednich przypadkach. Jest to spowodowane ograniczonymi zasobami, które nie są w stanie wesprzeć treningu lasu losowego złożonego z stu drzew decyzyjnych.

Reszta modelu pozostałą bez zmian by wyniki mimo zmniejszonej ilości drzew decyzyjnych nadal mogłyby być porównywane z poprzednimi wersjami lasu, nazwa modelu to model_random_forest_metod_i_długość_cięciwy_z_użyciem_końców
Oto jak wygląda skrypt:

```
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)

scaler=StandardScaler()
X_train=scaler.fit_transform(X_train)
X_test=scaler.transform(X_test)

model: RandomForestClassifier=RandomForestClassifier(n_estimators=50,random_state=42)
model.fit(X_train,y_train)

y_pred=model.predict(X_test)
acc=accuracy_score(y_test,y_pred)
print(f'Accuracy: {acc:.4f}')

labels=[1,2,3,4,5,6]
label_names=['Metoda 1-krótka','Metoda 1-długa','Metoda 2-krótka','Metoda 2-długa','
    Metoda 3-krótka','Metoda 3-długa']

cm=confusion_matrix(y_test,y_pred,labels=labels)
disp=ConfusionMatrixDisplay(confusion_matrix=cm,display_labels=label_names)
disp.plot()
plt.title('Macierz pomyłek dla modelu random_forest, metoda i długość cięciwy z uż
    yciem końcówek')
plt.tight_layout()
plt.savefig('macierz_pomyłek_dla_random_forest_metoda_i_długość_cięciwy_z_użyciem_koń
    cówek.png',dpi=300)
plt.show()

importance_of_features=model.feature_importances_
for f, imp in zip(features, importance_of_features):
    print(f'{f}: {imp:.3f}')
joblib.dump(model,'model_random_forest_metoda_i_długość_cięciwy_z_użyciem_końcówek.
    pkl')
```

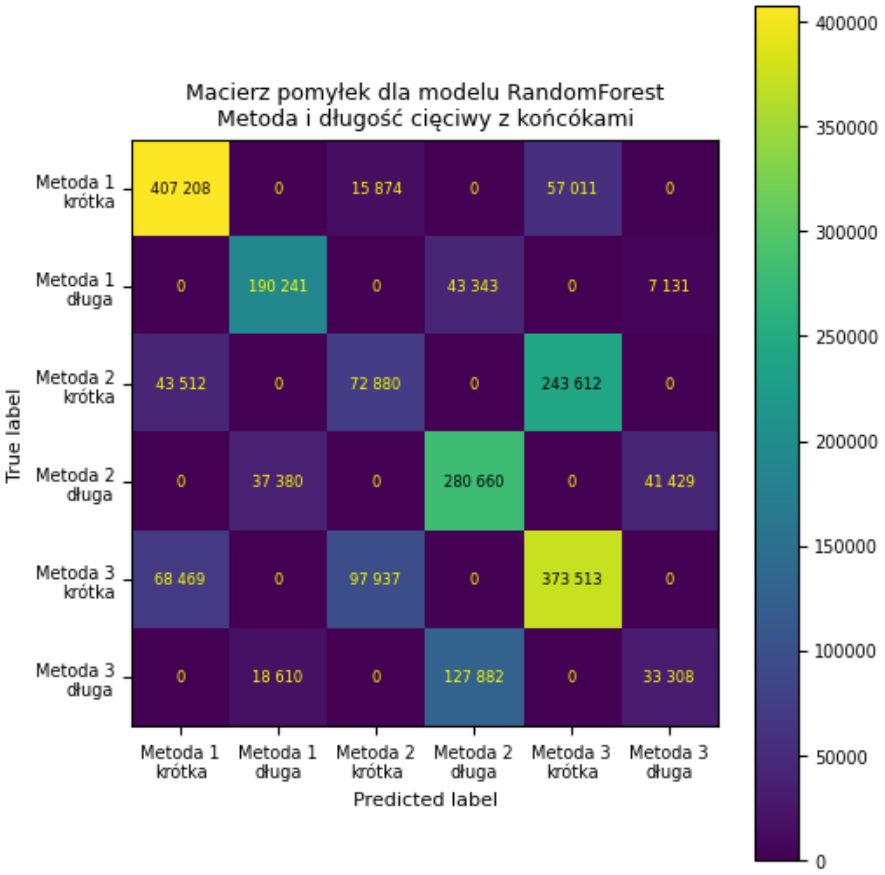
Ten las losowy osiągnął skuteczność na poziomie 64,6998%, co stanowi poprawę o około 10% w porównaniu do poprzednich wersji lasu losowego. Wynik ten przychyla nas do konkluzji, że wcześniejsze ucinanie danych mogło zatracać ważne z perspektywy modelu dane znajdujące się za daleko w danych by były one brane pod uwagę po skalowaniu.

Ważność cech w przypadku tego modelu wygląda następująco:

- x1: 0.05255930,
- y1: 0.05242327,
- x2: 0.05259569,
- y2: 0.05406845,
- length: 0.15370816,
- center_x: 0.05854644,

- center_y: 0.06190330,
- distance_from_center: 0.10661047,
- x1_koncowka: 0.06666388,
- y1_koncowka: 0.06658081,
- x2_koncowka: 0.06717485,
- y2_koncowka: 0.06729527,
- length_koncowka: 0.03902620,
- center_x_koncowka: 0.05095119,
- center_x_koncowka: 0.04989269,

a macierz pomyłek wyglądała następująco:



Rysunek 4.5: Macierz pomyłek dla modelu random forest z uwzględnieniem końcówek

Na podstawie powyższych wyników widać, że model ma rację w ponad połowie przypadków, cechy lenght oraz distance_from_center są najważniejszymi cechami dla modelu, a cechy zawierające w sobie skalowane wartości końcówek są dla lasu losowego ważniejsze niż oryginalne cechy.

Model ten najlepiej rozpoznaje cięciwy krótkie metody pierwszej i trzeciej oraz cięciwy długie metody drugiej. Jednakże model ma tendencję do przypisywania cięciw krótkich metody drugiej metodzie trzeciej, co jest jego cechą wspólną z modelem model_random_forest_metoda_i_długość_cięciwy.pkl.

4.3.2 Sieć neuronowa - klasyfikacja z użyciem końcówek

Podobnie jak w przypadku model_random_forest_metod_i_długość_cięciwy_z_użyciem_końcówek.pkl kod nie różni się zbytnio w porównaniu z poprzednią siecią neuronową model_cieć_neuronowa_i_długość_cięciwy.h5. Jedyną zmianą jest dostosowanie sieci do większej ilości klas po przez jej powiększenie o jedną warstwę Dense oraz o jedną warstwę Dropout. Kod wygląda następująco:

```
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)

scaler=StandardScaler()
X_train=scaler.fit_transform(X_train)
X_test=scaler.transform(X_test)

y_train_cat=tf.keras.utils.to_categorical(y_train-1,num_classes=6)
y_test_cat=tf.keras.utils.to_categorical(y_test-1,num_classes=6)

model=Sequential([
    Dense(64,activation='relu',input_shape=(X_train.shape[1],)),
    Dropout(0.2),
    Dense(32,activation='relu'),
    Dropout(0.2),
    Dense(16,activation='relu'),
    Dropout(0.2),
    Dense(8,activation='relu'),
    Dropout(0.2),
    Dense(6,activation='softmax')
])

model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

history=model.fit(X_train,y_train_cat, epochs=20,batch_size=32,validation_split=0.2,
    verbose=1,random_state=42)

loss,accuracy=model.evaluate(X_test,y_test_cat,verbose=0)
print(f'Dokładność na danych testowych: {accuracy:.4f}')
# Zapis dokładności do pliku
with open("dokladnosc_siec_neuronowa.txt", "w", encoding="utf-8") as f:
    f.write(f'Dokładność na danych testowych: {accuracy:.4f}\n')

y_pred_probs=model.predict(X_test)
y_pred=np.argmax(y_pred_probs,axis=1)
y_true=np.argmax(y_test_cat,axis=1)

labels=[0,1,2,3,4,5]
label_names=['Metoda 1-krótka','Metoda 1-długa','Metoda 2-krótka','Metoda 2-długa','
    Metoda 3-krótka','Metoda 3-długa']

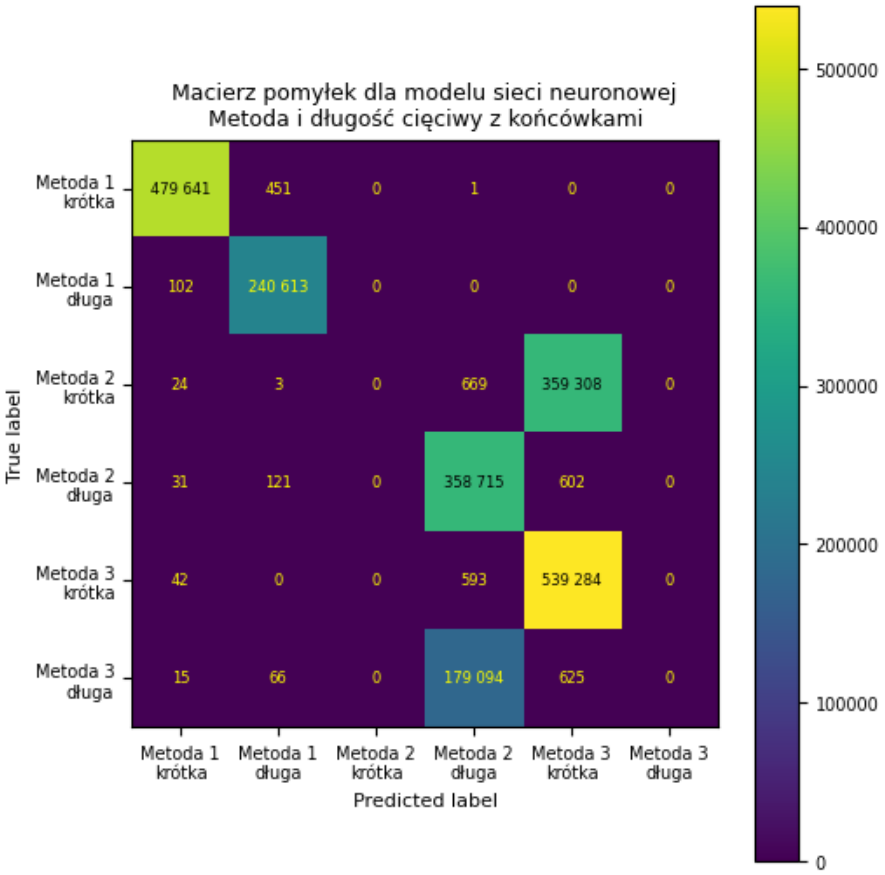
cm=confusion_matrix(y_true,y_pred,labels=labels)
disp=ConfusionMatrixDisplay(confusion_matrix=cm,display_labels=label_names)
disp.plot()
plt.title('Macierz pomyłek dla modelu sieci neuronowej, metoda i długość cięciwy z uż
    yciem końcówek')
plt.tight_layout()
plt.savefig('macierz_pomyłek_dla_sieci_neuronowej_metoda_i_długość_cięciwy_z_uż
    yciem_końcówek.png',dpi=300)
plt.show()
model.save("model_cieć_neuronowa_metoda_i_długość_cięciwy_z_użyciem_końcówek.h5")
joblib.dump(scaler, "scaler_do_model_cieć_neuronowa_metoda_i_długość_cięciwy_z_uż
    yciem_końcówek.pkl")
print("Model i scaler zapisane.")
```

Model zostaje zapisany w dwóch plikach:

- model_cieć_neuronowa_i_długość_cięciwy_z_użyciem_końcówek.h5 - wytrenowana sieć neuronowa,
- scaler_do_model_cieć_neuronowa_i_długość_cięciwy_z_użyciem_końcówek.pkl - obiekt skalera danych.

Sieć ta po treningu osiągnęła skuteczność na poziomie 74,91% co jest znaczącą poprawą skuteczności modelu w porównaniu z poprzednimi modelami, które nie osiągały wyniku 60%.

Macierz pomyłek dla tej sieci neuronowej wygląda następująco:



Rysunek 4.6: Macierz pomyłek dla modelu sieci neuronowej z uwzględnieniem końcówek

Sieć neuronowa rozpoznaje cięciwy wygenerowane przez metodę pierwszą niemalże perfekcyjnie. Widać także, że model bardzo dobrze nauczył się rozpoznawać cięciwy krótkie metody trzeciej, jak i cięciwy długie metody drugiej lecz prawie wszystkie cięciwy długie metody trzeciej przypisuje metodzie długiej, a cięciwy krótkie metody drugiej przypisuje metodzie trzeciej. Wynika to najpewniej z szansy na powstanie cięciwy o konkretnych długościach w zależności od metody.

4.3.3 Podsumowanie dzielenia z użyciem końcówek

Jak można było zaobserwować sieć neuronowa znowu poradziła sobie lepiej od lasu losowego (o około 10%), las losowy nadal ma problem z rozpoznawaniem cięciw wylosowanych za pomocą pierwszej metody, gdy sieć neuronowa robi to niemalże idealnie. Może to wynikać z tego, że metoda pierwsza korzysta z rozkładu jednostajnego opartego na punktach na okręgu, a nie jak metoda druga i trzecia w kole.

Las losowy nie popełnia, żadnych pomyłek jeśli chodzi o niepoprawne rozpoznawanie długości cięciwy, gdy sieć neuronowa ma ten problem. Głównym powodem tego jest najpewniej sama zasada działania obu modeli, gdzie to las losowy radzi sobie najlepiej z danymi które można podzielić na jasno określone kategorie.

Oba modele natomiast mają tendencję do przypisywanie cięciw krótkich drugiej metody trzeciej, oraz cięciw długich trzeciej metody drugiej.

Rozdział 5

Podsumowanie

Paradoks Bertranda pomimo swojej pozornej prostoty nadal jest omawiany przez matematyków, filozofów i informatyków. Opiera się on na tym jak cięciwy są losowane, z założeniem, że nie ważne od sposobu losowania tych że cięciw każda z nich ma równą szansę na bycie wylosowaną co w teorii powinno uniemożliwiać ich odróżnianie na sposób ich losowania. Jednakże na przestrzeni tej pracy ukazane zostało, że jest to jednak możliwe z użyciem modeli sztucznej inteligencji z zrozumieniem wad losowania cięciw za pomocą mocy obliczeniowej komputerów przy zaznaczeniu, że zwykłe komputery nie mają możliwości generowania liczb prawdziwie losowych.

Ukazane zostało także to, że w takich specyficznych zadaniach najlepiej radzą sobie modele które charakteryzują się wszechstronnością oraz elastycznością, czyli modele sieci neuronowych. Wyniki tych modeli mogą insynuować także to, że jednak nie jest tak, że każda cięciwa niezależnie od sposobu jej losowania ma równe prawdopodobieństwo wylosowania się co podważa sensowność jednego z założeń paradoksu Bertranda.

Co więcej ukazują one, że sieci neuronowe są w stanie znaleźć zależności w zbiorach danych, które dla człowieka wydawały by się bezsensowne. Stanowić o tym może umiejętność modelu model_cieć_neuronowa_i_długość_cięciwy _z_użyciem_końcówek do odróżnienia tego wedle jakiego rozkładu jednostajnego były losowane cięciwy, oraz że dane mogące zawierać dla modeli ważne informacje są utracane w skutek konwertowania danych na obsługiwane przez nie typy.

Sprawia to, że musimy się zastanowić czy przez przypadek w imię optymalizacji nie tracimy podczas treningu modeli SI części danych które mogą mieć znaczący wpływ na jakość ich wyników.

Bibliografia

[1] Bertrand, *Calcul des probabilités*, Gauthier-Villars, 1889.

[2] Edwin T. Jaynes, *The Well-Posed Problem*, Cambridge University Press, 2003.

[3] Aloy Drory, "Failure and Uses of Jaynes' Principle of Transformation Groups", *arXiv preprint*, 2016.
Dostępne online: `https://arxiv.org/pdf/1607.03751` (dostęp: 18 sierpnia 2025)

[4] Diederik Aerts, Massimiliano Sassoli de Bianchi, "Solving the hard problem of Bertrand's paradox", *arXiv preprint*, 2014.
Dostępne online: `https://arxiv.org/pdf/1403.4139` (dostęp: 18 sierpnia 2025)

[5] Pavel Brazhnikov, "Bertrand Paradox: Critical Consideration of Two of the Three Solutions in Terms of Geometry", *Preprints*, 2023.
Dostępne online: `https://www.preprints.org/manuscript/202303.0154` (dostęp: 18 sierpnia 2025)

[6] Javier Rodrigo, Mariló López, Sagrario Lantarón, "New Ways to Calculate the Probability in the Bertrand Problem", *ResearchGate*, 2023.
Dostępne online: `https://www.researchgate.net/publication/376667527_New_Ways_to_Calculate_the_Probability_in_the_Bertrand_Problem` (dostęp: 18 sierpnia 2025)

[7] TensorFlow Developers, "Keras Optimizers API".
Dostępne online: `https://www.tensorflow.org/api_docs/python/tf/keras/optimizers` (dostęp: 18 sierpnia 2025)

[8] Donald E. Knuth, *The Art of Computer Programming, Vol. 2: Seminumerical Algorithms*, 3rd ed., Addison-Wesley, 1997.